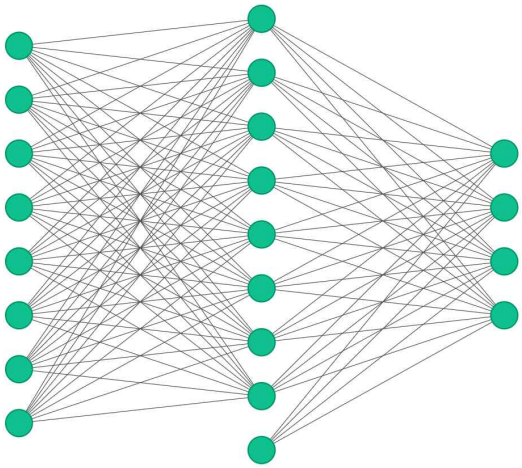
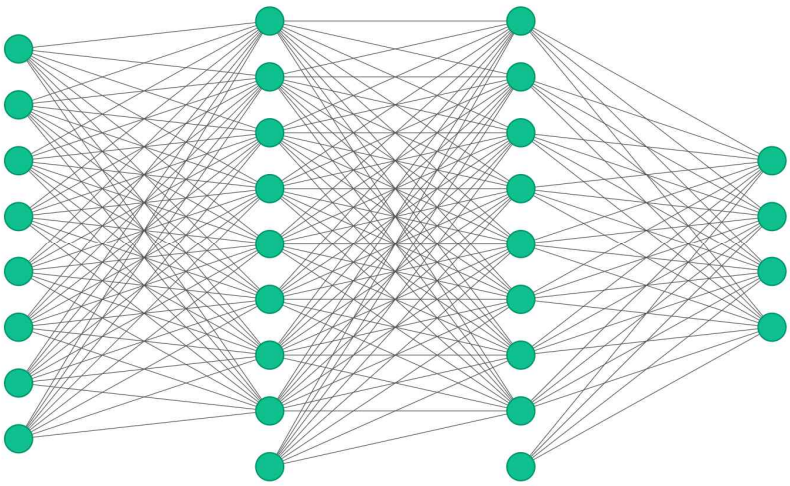
Chapter 04
인공 신경망 라이브러리 구현과 활용

01 인공 신경망 라이브러리 구현하기

여기서는 인공 신경망을 확장할 수 있도록 인공 신경망 라이브러리를 구현해 봅니다. 인공 신경망 라이브러리를 이용하면, 인공 신경망을 좀 더 자유롭게 구성하고 테스트해 볼 수 있습니다. 예를 들어, 다음과 같은 형태의 인공 신경망을 구성해서 테스트해 볼 수 있습니다.



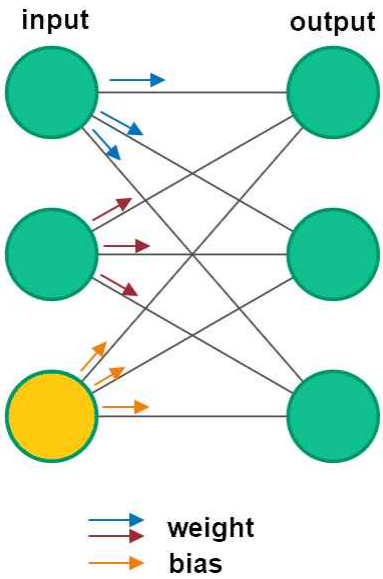
<7개의 입력, 8개의 은닉층, 4개의 출력층, 편향 포함>



<7개의 입력, 8개의 은닉층(1차), 8개의 은닉층(2차), 4개의 출력층, 편향 포함>

01 배열을 이용한 순전파 구현하기

여기서는 배열을 이용하여 인공 신경망으로 연결된 임의의 2 층간의 순전파를 구현해 봅니다. 다음과 같이 입력노드는 2개, 출력노드는 3개이며, 입력단에는 가중치가 포함됩니다. 노란색 노드는 가중치를 의미합니다.



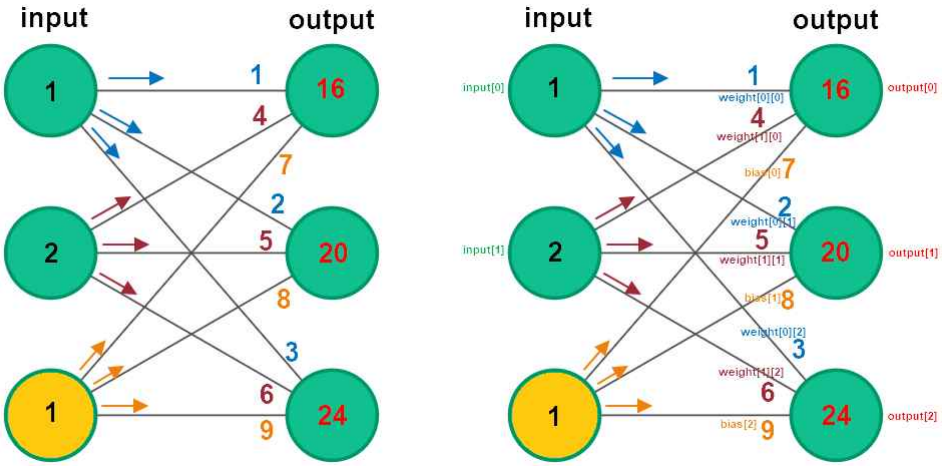
1. 다음과 같이 예제를 작성합니다.

421_1.ino

```
03      double input[2] = {1,2};
04      double output[3];
05      double weight[2][3] = {{1,2,3},{4,5,6}};
06      double bias[3] = {7,8,9};
07
08      void setup() {
09
10              Serial.begin(115200);
11              delay(1000);
12
13              for(int j=0;j<3;j++) {
14                      double sum = 0.0;
15                      for(int i=0;i<2;i++) {
16                              sum += input[i]*weight[i][j];
17                      }
18                      sum += 1*bias[j];
19                      output[j] = sum;
20                      printf("sum = %f\n", sum);
```

```
21                }
22        }
23
24        void loop() {
25
26        }
```

3 : 2개의 실수로 구성된 일차 배열 변수 input을 선언합니다. 값은 1, 2로 초기화합니다. 아래 그림을 참조합니다.
4 : 3개의 실수로 구성된 일차 배열 변수 output을 선언합니다. 아래 그림을 참조합니다.
5 : 입력 2개, 출력 3개를 연결할 수 있는 이차 배열 변수 weight를 선언합니다. weight 배열 변수는 3개의 실수로 구성된 일차 배열 2개로 구성된 이차 배열입니다.
6 : 3개의 실수로 구성된 일차 배열 변수 bias를 선언합니다.
10 : Serial.begin 함수를 호출하여 시리얼을 초기화하고 통신속도를 115200으로 설정합니다.
11 : delay 함수를 호출하여 1초간 지연을 줍니다. 시리얼 초기화가 될 때까지 충분한 시간을 줍니다.
13 : j 변수 0에서 2에 대하여 14~21줄을 수행합니다.
14 : 실수 변수 sum을 선언한 후, 0.0으로 초기화합니다.
15 : i 변수 0에서 1에 대하여 16줄을 수행합니다.
16 : input[i] 항목과 weight[i][j] 항목을 곱하여 sum에 더해 줍니다.



예를 들어, j가 0일 경우, i는 0, 1에 대해 (1*1 + 2*4)가 되어 sum값은 9가 됩니다. 위 그림을 참조합니다.
18 : bias[j] 항목에 1을 곱해 sum에 더해 줍니다. 예를 들어, j가 0일 경우, sum값은 기존에 계산된 값 9에 7이 더해져 16이 됩니다. 위 그림을 참조합니다.
19 : output[j] 항목에 sum 값을 옮깁니다. j가 0일 경우 output[j] 값은 16이 됩니다.
20 : printf 함수를 호출하여 sum값을 출력합니다.

2. [툴] 메뉴를 이용하여 보드, 포트를 다음과 같이 선택합니다.



3. 컴파일과 업로드를 수행합니다.





4. [시리얼 모니터] 버튼을 클릭합니다.



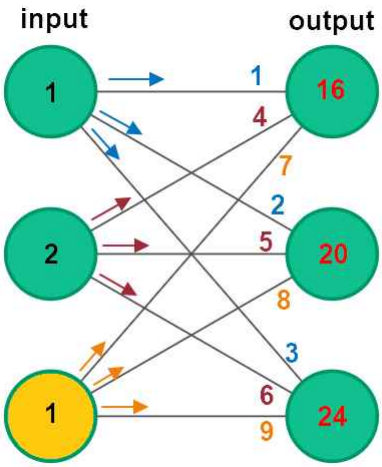
5. 시리얼 모니터 창이 뜨면, 우측 하단에서 통신 속도를 115200으로 맞춰줍니다.



6. 출력결과를 확인합니다.

```
sum = 16.000000
sum = 20.000000
sum = 24.000000
```

다음 그림을 참조합니다.



노드 개수 일반화하기

여기서는 노드의 개수를 변경하기 편하도록 일반화합니다.

1. 다음과 같이 예제를 수정합니다.

421_2.ino

```
03      const int INPUT_NODES = 2;
04      const int OUTPUT_NODES = 3;
05
06      double input[INPUT_NODES] = {1,2};
07      double output[OUTPUT_NODES];
08      double weight[INPUT_NODES][OUTPUT_NODES] = {{1,2,3},{4,5,6}};
09      double bias[OUTPUT_NODES] = {7,8,9};
10
11      void setup() {
12
13              Serial.begin(115200);
14              delay(1000);
15
16              for(int j=0;j<OUTPUT_NODES;j++) {
17                      double sum = 0.0;
18                      for(int i=0;i<INPUT_NODES;i++) {
19                              sum += input[i]*weight[i][j];
20                      }
21                      sum += 1*bias[j];
22                      output[j] = sum;
23                      printf("sum = %f\n", sum);
24              }
25      }
26
27      void loop() {
28
29      }
```

3 : 정수 상수 INPUT_NODES를 선언한 후, 2로 초기화합니다. 나중에 필요하면 초기값을 변경하여 노드의 개수를 조정할 수 있습니다.

4 : 정수 상수 OUTPUT_NODES를 선언한 후, 3으로 초기화합니다. 나중에 필요하면 초기값을 변경하여 노드의 개수를 조정할 수 있습니다.

6~9 : 배열 선언에서 항목의 개수를 나타내는 숫자를 INPUT_NODES, OUTPUT_NODES로 변경해 줍니다.

16, 18 : for 문의 상한 숫자를 INPUT_NODES, OUTPUT_NODES로 변경해 줍니다.

2. [툴] 메뉴를 이용하여 보드, 포트를 다음과 같이 선택합니다.



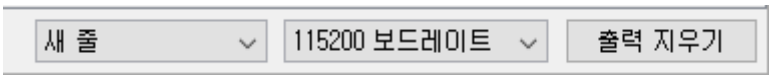
3. 컴파일과 업로드를 수행합니다.



4. [시리얼 모니터] 버튼을 클릭합니다.



5. 시리얼 모니터 창이 뜨면, 우측 하단에서 통신 속도를 115200으로 맞춰줍니다.



6. 출력결과를 확인합니다. 결과는 이전과 같습니다.

```
sum = 16.000000
sum = 20.000000
sum = 24.000000
```

순전파 함수 구현하기 1

여기서는 순전파 함수를 구현합니다. 순전파 함수를 구현하면 1단계 이상에서 반복적으로 호출하여 사용할 수 있습니다.

1. 다음과 같이 예제를 수정합니다.

421_3.ino

```
03      const int INPUT_NODES = 2;
04      const int OUTPUT_NODES = 3;
05
06      double input[INPUT_NODES] = {1,2};
07      double output[OUTPUT_NODES];
```

```
08      double weight[INPUT_NODES][OUTPUT_NODES] = {{1,2,3},{4,5,6}};
09      double bias[OUTPUT_NODES] = {7,8,9};
10
11      void feed_forward(
12      const double input[INPUT_NODES],
13      const double weight[INPUT_NODES][OUTPUT_NODES],
14      const double bias[OUTPUT_NODES],
15      double output[OUTPUT_NODES],
16      const int INPUT_NODES,
17      const int OUTPUT_NODES) {
18              for(int j=0;j<OUTPUT_NODES;j++) {
19                      double sum = 0.0;
20                      for(int i=0;i<INPUT_NODES;i++) {
21                              sum += input[i]*weight[i][j];
22                      }
23                      sum += 1*bias[j];
24                      output[j] = sum;
25                      printf("sum = %f\n", sum);
26              }
27      }
28
29      void setup() {
30              Serial.begin(115200);
31              delay(1000);
32
33              feed_forward(input,
34              weight,
35              bias,
36              output,
37              INPUT_NODES,
38              OUTPUT_NODES);
39      }
40
41      void loop() {
42
43      }
```

11~27 : 순전파 함수 feed_forward를 구현합니다.

12 : 입력층 일차 배열을 받습니다. const 키워드로 상수 속성을 주어 함수 내부에서 변경하지 못하게 합니다.

13 : 가중치 이차 배열을 받습니다. const 키워드로 상수 속성을 주어 함수 내부에서 변경하지 못하게 합니다.

14 : 편향 일차 배열을 받습니다. const 키워드로 상수 속성을 주어 함수 내부에서 변경하지 못하게 합니다.
15 : 출력층 일차 배열을 받습니다.
16 : 입력층 일차 배열의 항목 개수를 받습니다.
17 : 출력층 일차 배열의 항목 개수를 받습니다.
18~26 : 이전에 구현한 예제와 내용이 같습니다.
33~38 : 순전파 함수 feed_forward 함수를 호출합니다. 첫 번째 인자로 input 배열, 두 번째 인자로 weight, 세 번째 인자로 bias, 네 번째 인자로 output, 다섯 번째 인자로 INPUT_NODES, 마지막 인자로 OUTPUT_NODES를 넘겨줍니다.

2. [툴] 메뉴를 이용하여 보드, 포트를 다음과 같이 선택합니다.



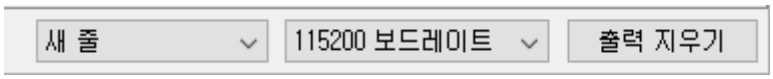
3. 컴파일과 업로드를 수행합니다.



4. [시리얼 모니터] 버튼을 클릭합니다.



5. 시리얼 모니터 창이 뜨면, 우측 하단에서 통신 속도를 115200으로 맞춰줍니다.



6. 출력결과를 확인합니다. 결과는 이전과 같습니다.

```
sum = 16.000000
sum = 20.000000
sum = 24.000000
```

순전파 함수 구현하기 2

여기서는 순전파 함수의 매개변수 부분을 다듬습니다. 배열 표현 형식을 포인터 표현 형식으

로 변경해 줍니다.

1. 다음과 같이 예제를 수정합니다.

421_4.ino

```
03      const int INPUT_NODES = 2;
04      const int OUTPUT_NODES = 3;
05
06      double input[INPUT_NODES] = {1,2};
07      double output[OUTPUT_NODES];
08      double weight[INPUT_NODES][OUTPUT_NODES] = {{1,2,3},{4,5,6}};
09      double bias[OUTPUT_NODES] = {7,8,9};
10
11      void feed_forward(
12      const double *input,
13      const double (* weight)[OUTPUT_NODES],
14      const double *bias,
15      double *output,
16      const int INPUT_NODES,
17      const int OUTPUT_NODES) {
18              for(int j=0;j<OUTPUT_NODES;j++) {
19                      double sum = 0.0;
20                      for(int i=0;i<INPUT_NODES;i++) {
21                              sum += input[i]*weight[i][j];
22                      }
23                      sum += 1*bias[j];
24                      output[j] = sum;
25                      printf("sum = %f\n", sum);
26              }
27      }
28
29      void setup() {
30              Serial.begin(115200);
31              delay(1000);
32
33              feed_forward(input,
34              weight,
35              bias,
36              output,
37              INPUT_NODES,
38              OUTPUT_NODES);
```

```
39      }
40
41      void loop() {
42
43      }
```

12, 14, 15 : 배열 표현법을 포인터 표현법으로 바꿔줍니다. 함수의 매개 변수에 표현된 배열은 실제로 포인터로 해석되어 처리됩니다. 매개 변수 자리에 온 일차 배열은 일차 포인터와 같습니다. 역으로 매개 변수 자리에 온 일차 포인터는 일차 배열을 받습니다.
13 : 배열 표현법을 포인터 표현법으로 바꿔줍니다. 함수의 매개 변수에 표현된 배열은 실제로 포인터로 해석되어 처리됩니다. 매개 변수 자리에 온 이차 배열은 일차 배열을 가리키는 포인터와 같습니다. 표현법이 조금 복잡합니다. 매개 변수 자리에서 이차 배열을 포인터 표현법으로 바꾸고자 할 경우엔 일차 배열의 개수는 꼭 필요합니다. 여기서는 OUTPUT_NODES가 명시되어야 합니다. 자세한 내용은 C 포인터를 참고하기 바랍니다.

2. [툴] 메뉴를 이용하여 보드, 포트를 다음과 같이 선택합니다.



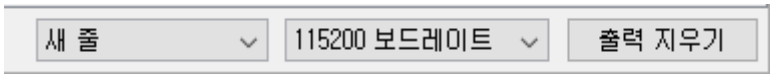
3. 컴파일과 업로드를 수행합니다.



4. [시리얼 모니터] 버튼을 클릭합니다.



5. 시리얼 모니터 창이 뜨면, 우측 하단에서 통신 속도를 115200으로 맞춰줍니다.



6. 출력결과를 확인합니다. 결과는 이전과 같습니다.

```
sum = 16.000000
sum = 20.000000
sum = 24.000000
```

순전파 함수 구현하기 3

여기서는 순전파 함수의 매개변수 부분을 다듬습니다. 일차 배열의 포인터 표현 형식을 포인터 표현 형식으로 변경해 줍니다. 출력 노드의 개수를 일반화해주기 위해 이 과정이 필요합니다. 뒤에서 우리는 입력 노드의 개수와 출력 노드의 개수를 자유롭게 바꿔서 테스트하게 됩니다.

1. 다음과 같이 예제를 수정합니다.

421_5.ino

```
03      const int INPUT_NODES = 2;
04      const int OUTPUT_NODES = 3;
05
06      double input[INPUT_NODES] = {1,2};
07      double output[OUTPUT_NODES];
08      double weight[INPUT_NODES][OUTPUT_NODES] = {{1,2,3},{4,5,6}};
09      double bias[OUTPUT_NODES] = {7,8,9};
10
11      void feed_forward(
12      const double *input,
13      const double * weight,
14      const double *bias,
15      double *output,
16      const int INPUT_NODES,
17      const int OUTPUT_NODES) {
18              for(int j=0;j<OUTPUT_NODES;j++) {
19                      double sum = 0.0;
20                      for(int i=0;i<INPUT_NODES;i++) {
21                              sum += input[i]*weight[i*OUTPUT_NODES+j];
22                      }
23                      sum += 1*bias[j];
24                      output[j] = sum;
25                      printf("sum = %f\n", sum);
26              }
27      }
28
29      void setup() {
30              Serial.begin(115200);
31              delay(1000);
```

```
32
33              feed_forward(input,
34              (const double *)weight,
35              bias,
36              output,
37              INPUT_NODES,
38              OUTPUT_NODES);
39      }
40
41      void loop() {
42
43      }
```

13 : 실수 일차 배열의 포인터 표현법을 실수 포인터 표현법으로 바꿔줍니다. 이것은 이차 배열을 일차 배열로 받는 방법입니다. 이렇게 하면 출력 노드의 개수 OUTPUT_NODES를 명시할 필요가 없습니다.

21 : weight 배열의 항목을 나타내는 부분을 변경해 줍니다.

34 : feed_forward 함수 호출시, 2번째 인자를 일차 배열로 형변환 해 줍니다.

2. [툴] 메뉴를 이용하여 보드, 포트를 다음과 같이 선택합니다.



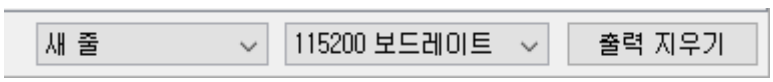
3. 컴파일과 업로드를 수행합니다.



4. [시리얼 모니터] 버튼을 클릭합니다.



5. 시리얼 모니터 창이 뜨면, 우측 하단에서 통신 속도를 115200으로 맞춰줍니다.



6. 출력결과를 확인합니다. 결과는 이전과 같습니다.

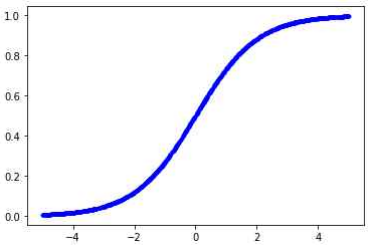
```
sum = 16.000000
sum = 20.000000
sum = 24.000000
```

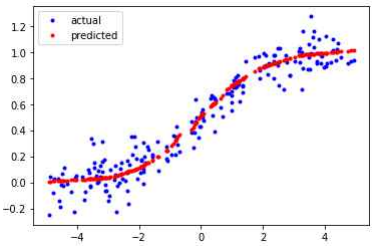
03 활성화 함수 구현하기

여기서는 출력노드에 활성화 함수를 추가해 봅니다. 역입력노드에도 역할성화 함수를 추가합니다. 활성화 함수로는 sigmoid, relu 함수를 추가해 봅니다. 우리는 이전 단원에서 sigmoid를 활성화함수로 사용하는 신경망을 구현해 보았습니다.

sigmoid 함수

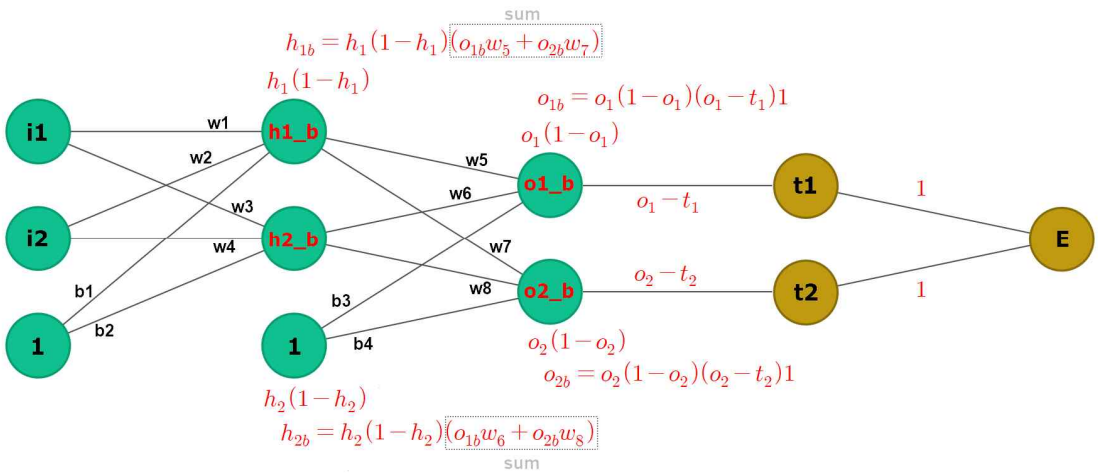
다음은 sigmoid 함수에 대한 그래프와 인공 신경망 학습 후, 예측 그래프입니다. 우리는 앞에서 이 그래프를 직접 그려보았습니다.





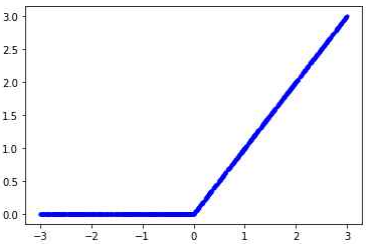
$$y = \frac{1}{1-e^{-x}} \ (-5 \le x \le 5)$$

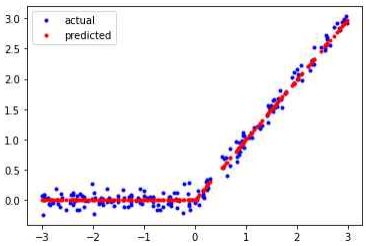
다음 그림과 같이 sigmoid 함수의 역전파 과정도 살펴보았습니다. 그리고 순전파 과정에서의 출력값을 역전파 과정에서 사용하는 것도 살펴보았습니다.



relu 함수

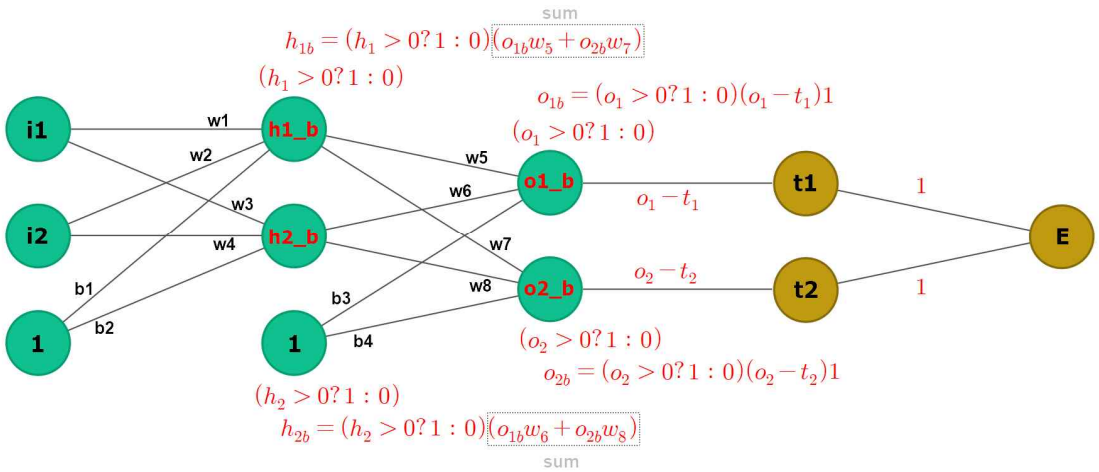
다음은 relu 함수에 대한 그래프와 인공 신경망 학습 후, 예측 그래프입니다. 우리는 앞에서 이 그래프를 직접 그려보았습니다.





$$y = \begin{cases} x\,(x > 0) \\ 0\,(x \le 0) \end{cases} (-3 \le x \le 3)$$

다음 그림은 relu 함수의 역전파를 나타내는 그림입니다.



relu 함수의 경우 순전파시 0보다 큰 값은 그대로 출력하고 0보다 작거나 같은 값은 0을 출력합니다. 즉, 0보다 큰 신호만 유효 신호로 처리합니다. 순전파 때 0보다 큰 값으로 출력된 신호에 대해서만 역전파시 해당 오차를 받게됩니다. 순전파 때 영향을 준 출력에 대해서만 역전파 때 유효한 오차로 계산하게 됩니다. 즉, 0으로 나간 신호에 대해서는 오차도 0으로 계산되게 됩니다.

1. 다음과 같이 예제를 수정합니다.

423_1.ino

```
01      #define printf _printf
02
03      const int INPUT_NODES = 2;
04      const int OUTPUT_NODES = 3;
05
```

```
06      double input[INPUT_NODES] = {1,2};
07      double output[OUTPUT_NODES];
08      double weight[INPUT_NODES][OUTPUT_NODES] = {{1,2,3},{4,5,6}};
09      double bias[OUTPUT_NODES] = {7,8,9};
10
11      typedef enum _activation {
12              LINEAR=0,
13              SIGMOID=1,
14              RELU=2,
15      } activation_t;
16
17      void feed_forward(
18      const double *input,
19      const double * weight,
20      const double *bias,
21      double *output,
22      const int INPUT_NODES,
23      const int OUTPUT_NODES,
24      activation_t activation) {
25              for(int j=0;j<OUTPUT_NODES;j++) {
26                      double sum = 0.0;
27                      for(int i=0;i<INPUT_NODES;i++) {
28                              sum += input[i]*weight[i*OUTPUT_NODES+j];
29                      }
30                      sum += 1*bias[j];
31                      if(activation==LINEAR) {
32                              output[j] = sum;
33                      } else if(activation==SIGMOID) {
34                              output[j] = 1.0/(1.0+exp(-sum));
35                      } else if(activation==RELU) {
36                              output[j] = sum>0?sum:0;
37                      }
38                      printf("output[%d] = %f\n", j, output[j]);
39              }
40      }
68
69      void setup() {
70              Serial.begin(115200);
71              delay(1000);
72
```

```
73                  feed_forward(input,
74                  (const double *)weight,
75                  bias,
76                  output,
77                  INPUT_NODES,
78                  OUTPUT_NODES,
79                  SIGMOID);
80
92      }
93
94      void loop() {
95
96      }
```

11~15 : 활성화 함수의 종류를 나타내는 열거형 상수 activation_t을 정의합니다. activation_t 형의 값으로는 LINEAR, SIGMOID, RELU를 정의합니다. LINEAR는 활성화함수를 따로 쓰지 않는 경우를 의미합니다.
24 : feed_forward 함수에 activation_t 매개변수를 추가합니다.
31~37 : 매개변수를 통해 넘어온 활성화 함수의 종류에 따라 output[j]값을 계산합니다.
38 : printf 함수를 호출하여 output[j] 값을 출력합니다.
48 : back_propagation 함수에 순전파 때의 입력층 노드값 배열 매개변수를 추가합니다. 입력층 노드값 배열 매개변수는 역전파 때에 역입력층의 노드값 계산에 사용됩니다.
52 : back_propagation 함수에 activation_t 매개변수를 추가합니다.
58~64 : 매개변수를 통해 넘어온 활성화 함수의 종류에 따라 input_b[j]값을 계산합니다.
65 : printf 함수를 호출하여 input_b[j] 값을 출력합니다.
79 : feed_forward 함수 호출 부분에 마지막 인자를 추가합니다. 여기서는 SIGMOID로 선택합니다.

2. [툴] 메뉴를 이용하여 보드, 포트를 다음과 같이 선택합니다.



3. 컴파일과 업로드를 수행합니다.

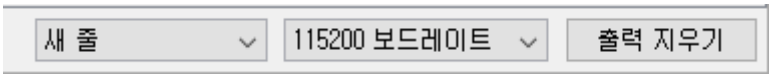




4. [시리얼 모니터] 버튼을 클릭합니다.



5. 시리얼 모니터 창이 뜨면, 우측 하단에서 통신 속도를 115200으로 맞춰줍니다.



6. 출력결과를 확인합니다.

```
output[0] = 1.000000
output[1] = 1.000000
output[2] = 1.000000
input_b[0] = 0.000000
input_b[1] = -30.000000
```